2 - Implement RK4

Use baseline timestep $\Delta t = 0.01$ and $T = 50$.

a. Run code with $\Delta t, \Delta t/2, \Delta t/4$.

b. Plot $x(t)$ for each run on the same graph. Comment on convergence with decreasing Δt.

c. Use built-in ODE solver and compare results.

Solution.

The following code was used to implement RK4 and run comparison with SciPy's built-in ODE solver, `solve_ivp` (similar to MATLAB's `ode45`):

Listing 1: Question 2 Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def F(u, sigma, rho, beta):
    x, y, z = u
    dxdt = sigma * (y - x)
    dydt = x * (rho - z) - y
    dzdt = x * y - beta * z
    return np.array([dxdt, dydt, dzdt])

def RK4_step(u, h, sigma, rho, beta):
    k1 = F(u, sigma, rho, beta)
    k2 = F(u + (h / 2) * k1, sigma, rho, beta)
    k3 = F(u + (h / 2) * k2, sigma, rho, beta)
    k4 = F(u + h * k3, sigma, rho, beta)

    y_new = u + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)
    return y_new

def RK4_solver(u_0, h, sigma, rho, beta, T=50):
    num_steps = int(T / h)
    u_values = np.zeros((num_steps + 1, len(u_0)))
    u_values[0] = u_0

    for n in range(num_steps):
        u_values[n + 1] = RK4_step(u_values[n], h, sigma, rho, beta)

    return u_values
```

```python

if __name__ == "__main__":
    # < Some Styling Setup and Enable Latex >
    # ...
    # Use default parameters for Lorenz system (Case C)
    sigma, rho, beta = 10, 28, 8/3
    u_0 = np.array([1, 1, 1])
    dt = 0.01
    time_steps = [dt, dt/2, dt/4]

    # Run RK4 solver for each time step and plot results
    for h in time_steps:
        u_values = RK4_solver(u_0, h, sigma, rho, beta)
        time_values = np.arange(0, 50 + h, h)
        plt.plot(time_values, u_values[:, 0], label=rf"$\Delta t = {h}$")
    plt.xlabel(r"$t$")
    plt.ylabel(r"$x(t)$")
    plt.title("RK4 Results")
    plt.legend()
    plt.savefig('./outputs/rk4_q2.png', dpi=300)
    plt.cla()

    # Compare with SciPy's ODE solver
    scipy_solution = solve_ivp(
        lambda t, u: F(u, sigma, rho, beta),
        [0, 50],
        u_0,
        t_eval=np.arange(0, 50 + dt, dt)
    )

    # Plot Scipy solution
    plt.plot(scipy_solution.t, scipy_solution.y[0], label="Scipy", color=
   default_colors[3])
    plt.xlabel(r"$t$")
    plt.ylabel(r"$x(t)$")
    plt.title("Scipy Solver Results")
    plt.legend()
    plt.savefig('./outputs/rk4_scipy.png', dpi=300)
    plt.cla()
```

The following plots were obtained:

Figure 1: RK4 results for different time steps.

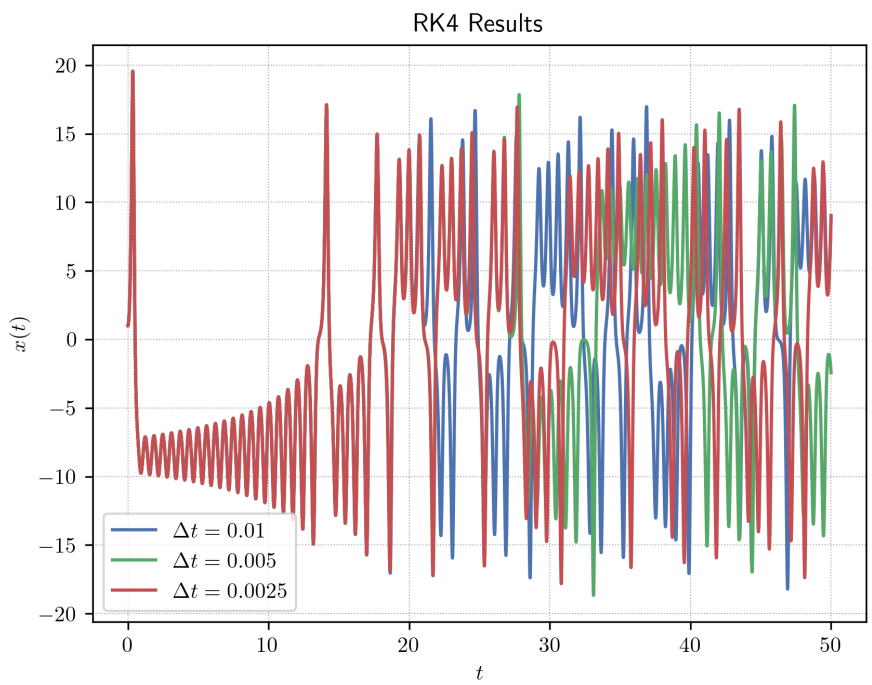
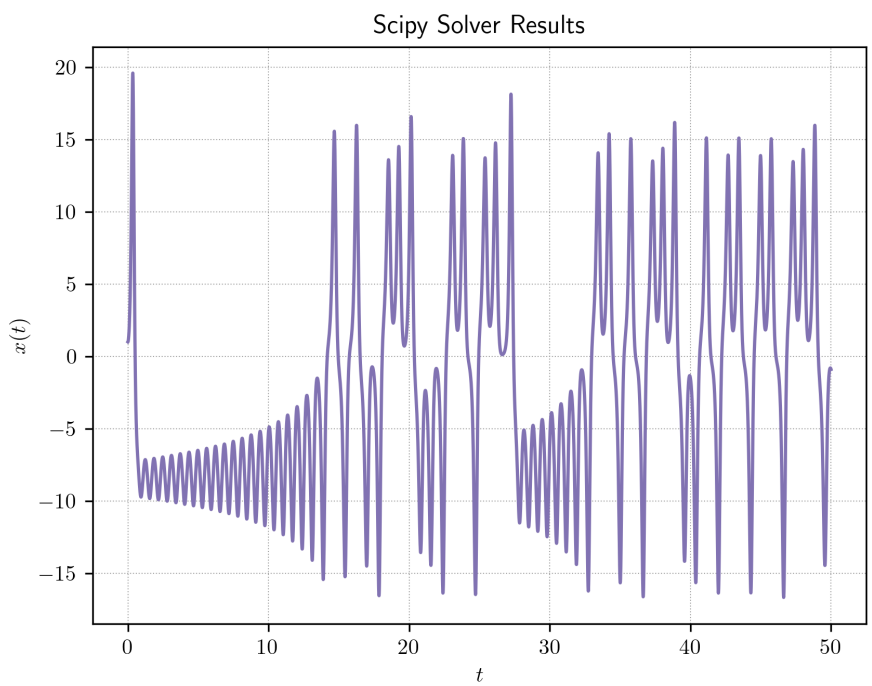


Figure 2: SciPy's `solve_ivp` results.



From Figure 1, we observe that as the time step Δt decreases, the solution seems to converge to a more consistent trajectory for larger time intervals. Or in other words, we see that the overlap for trajectories with smaller times steps is better before the Lorenz system's chaotic nature causes divergence.

In comparing Figures 1 and 2, we can see that the RK4 implementation closely matches the results from SciPy's `solve_ivp` up until about $t \approx 13$ (when graphs are overlapped). After this point, the trajectories begin to diverge due to numerical differences. This behavior is expected as `solve_ivp` uses an adaptive time-stepping method in comparison to our fixed time step RK4 implementation, which can lead to different numerical errors accumulating over time. However, since we observed that the initial trajectories match closely, we can conclude that our RK4 implementation is correct and captures the dynamics of the Lorenz system accurately for smaller time intervals.

---

3 - Parameter Exploration

Simulate $[0, T = 50]$ for the following parameter sets:

Case A:$(\sigma, \rho, \beta) = (10, 5, 8/3)$ **Case B:**$(\sigma, \rho, \beta) = (10, 15, 2)$

Case C:$(\sigma, \rho, \beta) = (10, 28, 8/3)$ **Case D:**$(\sigma, \rho, \beta) = (10, 40, 8/3)$

For each case:

a. Plot $x(t)$, $y(t)$, and $z(t)$ vs. t over $[0, T]$.

b. Plot the trajectory $(x(t), y(t), z(t))$ in phase space.

c. Comment on the behaviors of the system for each case.

Solution. temp text

---

4 - Sensitivity Experiment (repeatability test)

Choose one of the four parameter sets from Question 3. Run two simulations with the following initial conditions:

$$(x_0, y_0, z_0) = (1, 1, 1) \quad (x_0, y_0, z_0) = (1 + 10^{-8}, 1, 1)$$

using the same Δt.

a. Plot $x(t)$ from both runs on the same graph. Comment on the results.

b. Plot the difference $|x_1(t) - x_2(t)|$ vs. t on a semilog-y plot (i.e., only the y-axis is logarithmic). Comment on the results.

c. Do the Same for $y(t)$ and $z(t)$.

Solution. temp text

---

5 - Time Step Effects and Numerical Stability

For Case D, repeat the computation with:

$$\Delta t = 0.02, 0.01, 0.005$$

a. Compare the time series plots and the phase-space trajectories.

b. Describe any visible artifacts (eg. drift, distortion of trajectories, blow-up)

c. Describe the influence of step size from the simulation results.

Solution. dummy text